Cover sheet for submission of work for assessment

UNIT DETAILS

Unit name	Big Data Architecture and Application			Class day/time	Mondays, 7:00 - 10:00
Unit code	COS20028	Assignment no.	01	Due date	Jun 14th
Name of lecturer/teacher	Mr. Doan Tan Trung				
Tutor/marker's name	Mr. Doan Tan Trung				

Office use only

Faculty or school date stamp

STUDENT(S)

	Family Name(s)	Given Name(s)	Student ID Number(s)
(1)	Truong	Hieu	105565520
(2)			
(3)			
(4)			
(5)			
(6)			

DECLARATION AND STATEMENT OF AUTHORSHIP

1. I/we have not impersonated, or allowed myself/ourselves to be impersonated by any person for the purposes of this assessment.
2. This assessment is my/our original work and no part of it has been copied from any other source except where due acknowledgement is made.
3. No part of this assessment has been written for me/us by any other person except where such collaboration has been authorised by the lecturer/teacher concerned.
4. I/we have not previously submitted this work for this or any other course/unit.
5. I/we give permission for my/our assessment response to be reproduced, communicated, compared and archived for plagiarism detection, benchmarking or educational purposes.

I/we understand that:

6. Plagiarism is the presentation of the work, idea or creation of another person as though it is your own. It is a form of cheating and is a very serious academic offence that may lead to exclusion from the University. Plagiarised material can be drawn from, and presented in, written, graphic and visual form, including electronic data and oral presentations. Plagiarism occurs when the origin of the material used is not appropriately cited.

Student signature/s

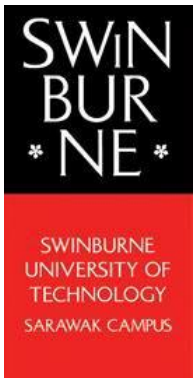
I/we declare that I/we have read and understood the declaration and statement of authorship.

(1) 

Further information relating to the penalties for plagiarism, which range from a formal caution to expulsion from the University is contained on the Current Students website at **www.swin.edu.au/student/**

Copies of this form can be downloaded from the Student Forms web page at **www.swinburne.edu.au/studentforms/**

PAGE 1 OF 1



Swinburne University of Technology Hawthorn Campus
Department of Computing Technologies

COS20028 Big Data Architecture and Application

Assignment 1 - *Semester 2, 2025*

Student Name: Truong Ngoc Gia Hieu
Student ID: SWS01217/ 105565520
Assessment Title: MapReduce Application.
Assessment Outcome: 20%
Due Date: AEST 23:59 on 5/09/2025.
Assessable Item:

- One (1) piece of a report containing the complete flowchart showing your design idea about how to prepare the data, the tools being used in all steps, and the report for answers to the given questions.
- One (1) document of the code collections for your assignment.

- For questions 1 to 3, build a MapReduce program with to find the word counts of words composed of at least three letters and exclude all words starting with numerical numbers.
  For example, if the input file contains "at home 013." The output of your MapReduce program should only contain "home 1"

Part 1:

1. List all commands used in this assignment in order, as well as readable execution screenshots with your name, student ID, and the VM datetime information with format "Program Executed at: yyyy-MM-dd HH:mm:ss" printed from your code. Treat the upper and lower case words independently. Moreover, answer the questions based on your retrieved result. ※Note: It is quite common if you see the VM's time is different from the real-world time. No need to adjust the VM system time to match reality. **[15 marks]**
2. Correct implementation of the Mapper class(es) with readability and well-structured codes and comments. **[20 marks]**
3. Correct implementation of the Reducer class(es) with readability and well-structured codes and comments. **[10 marks]**
4. Correct implementation of the Driver class(es) with readability and well-structured codes and comments. **[20 marks]**

Part 2:

5. Create a Java project named **Assignment1_2** and use the output from **Assignment1** as the input. Write a MapReduce code to count how many times a number appears. List all commands used in this assignment in order, as well as readable execution screenshots with your name, student ID, and the VM datetime information with format "Program Executed at: yyyy-MM-dd HH:mm:ss" printed from your code. The corresponding codes should also be packed in the submission. **[25 marks]**
6. Explain your chain of thought in solving the given tasks. Include a flowchart in the report to

explain the chain of thought step by step. **[10 marks]**

Part 1:

1. List the commands used in the assessment in order, as well as readable execution screenshots with your name, student ID, and the VM datetime information with format "Program Executed at: yyyy-MM-dd HH:mm:ss" printed from your code. Treat the upper and lower case words independently.

Ans:

1	pwd
2	javac -classpath `hadoop classpath` stubs/*.java
3	jar cvf assign1a.jar stubs/*.class
4	hadoop jar assign1a.jar stubs/Assign1Driver little_women assign1a
5	hdfs dfs -get assign1a
6	less assign1a/part-r-00000

- How many times does the word "Amy" appear?

  Ans: The word "Amy" appears 9 times.

- How many times does the word "coffee" appear?

  Ans: The word "Coffee" appear 1 time

2. Mapper Class:

Assign1Mapper.java | Assign1Reducer.java | Assign1Driver.java

```
1  //Import crucial libraries and tools. //
2  package stubs;
3
4  import java.util.Date;
5  import java.io.IOException;
6  import java.util.regex.Matcher;
7  import java.util.regex.Pattern;
8  import org.apache.hadoop.io.Text;
9  import java.text.SimpleDateFormat;
10 import org.apache.hadoop.io.IntWritable;
11 import org.apache.hadoop.io.LongWritable;
12 import org.apache.hadoop.mapreduce.Mapper;
13
14 public class Assign1Mapper extends Mapper<LongWritable, Text, Text, IntWritable> {
15     // Define the constant map output value (Writable Type).
16     private final static IntWritable ONE = new IntWritable(1);
17     private Text outputKey = new Text();
18
19     // Requirement 1: Use Regular Expressions to validate and filter tokens.
20     // 1.1 Matches words that start with a letter.
21     private static final Pattern ALPHABET_PATTERN = Pattern.compile("^[a-zA-Z]+");
22     private SimpleDateFormat outputFormat;
23
24     @Override
25     protected void setup (Context context) throws IOException, InterruptedException {
26         // Requirement 2: Format dates to Year-Month.
27         outputFormat = new SimpleDateFormat("yyyy-MM");
```

Figure 1: File Assign1Mapper.java (Part I)

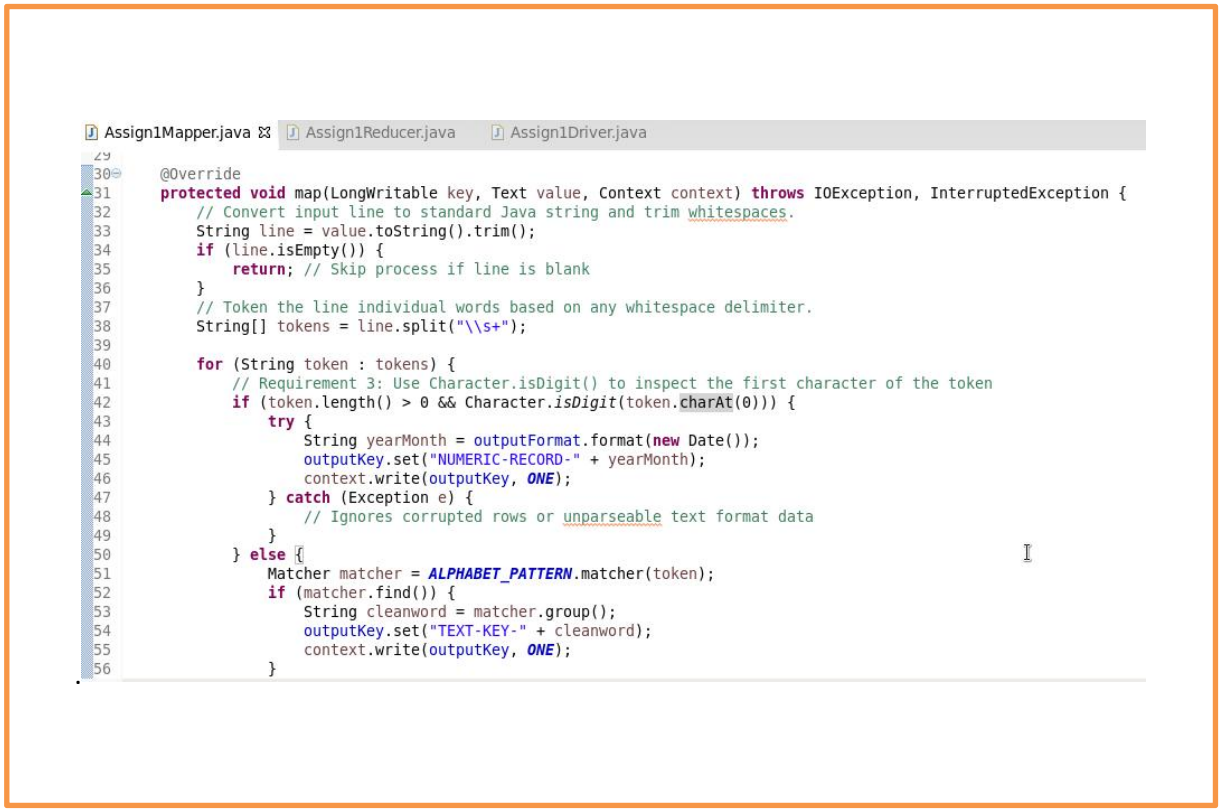


Figure 2: File Assign1Mapper.java (Part 2)

3. Reducer Class:

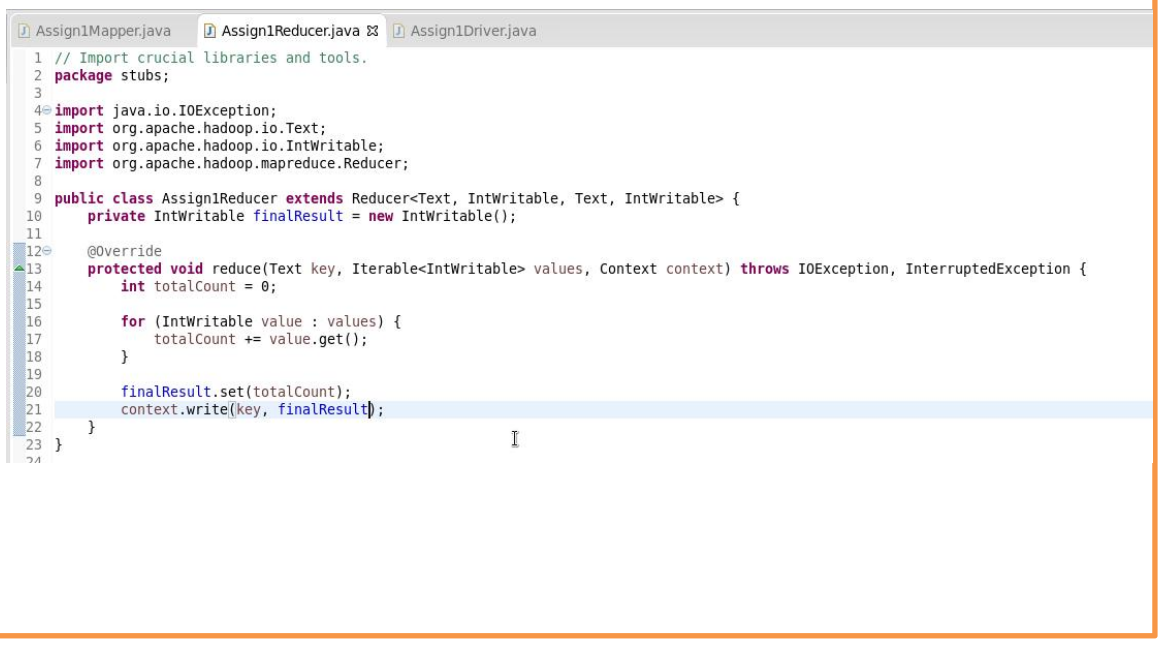


Figure 3: File Assign1Reducer.java

4. Driver Class:

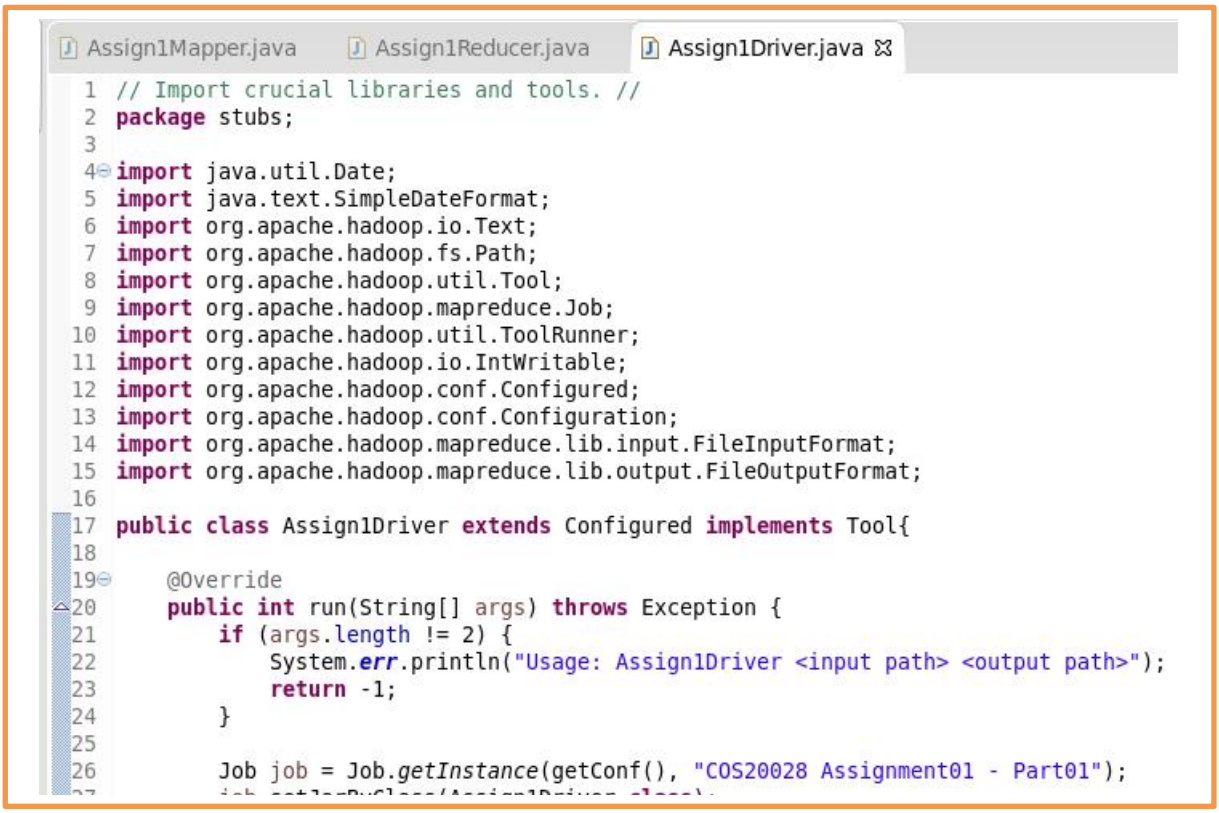


Figure 4: File Assign1Driver.java (Part I)

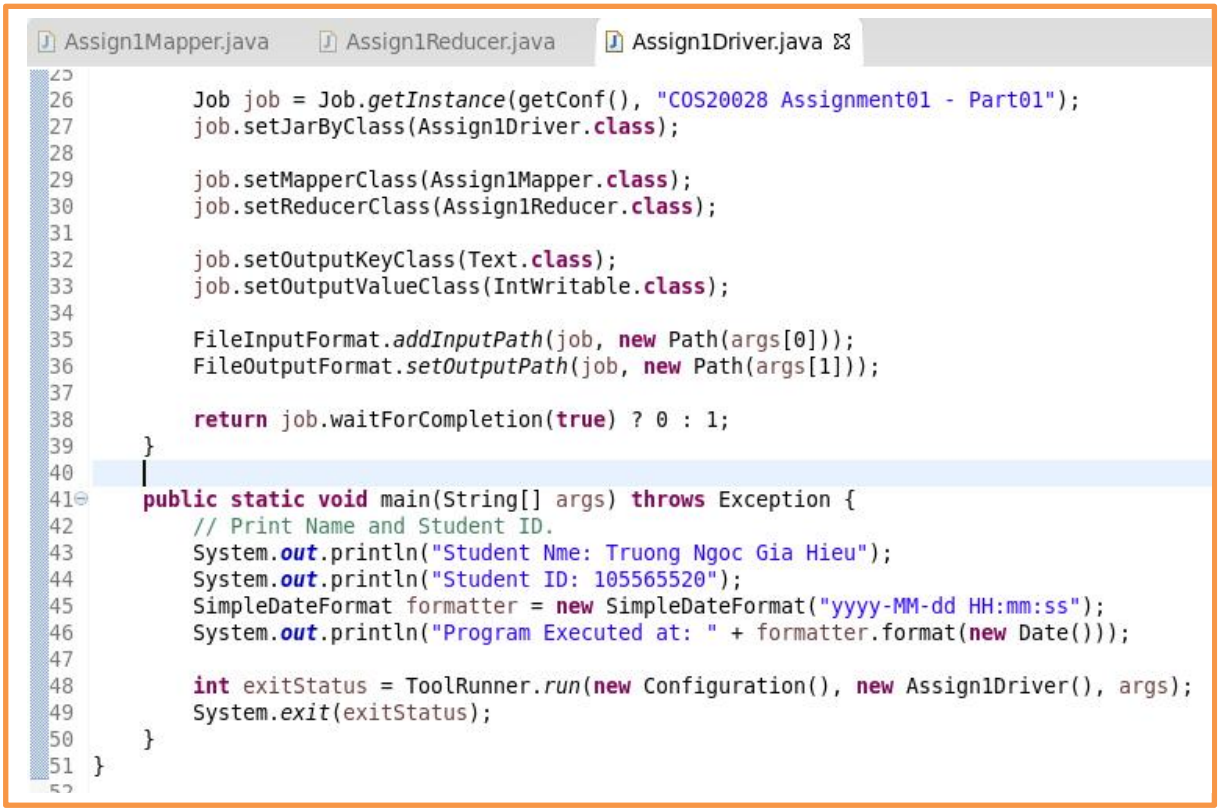


Figure 5: File Assign1Driver.java (Part 2)

Part 2

5. Create a Java project named **Assignment1_2** and use the output from **Assignment1** as the input. Write a MapReduce code to count how many times a number appears. List all commands used in this assignment in order, as well as readable execution screenshots with your name, student ID, and the VM datetime information with format "Program Executed at: yyyy-MM-dd HH:mm:ss" printed from your code.

Ans:

1	pwd
2	javac -classpath `hadoop classpath` stubs/*.java
3	jar cvf assign1b.jar stubs/*.class
4	hadoop jar assign1b.jar stubs/Assign1Driver assign1a assign1b
5	hdfs dfs -get assign1b
6	Less assign1b/part-r-00000

- What is the most frequently occurring word count in the input file?

  Ans: The most frequently occurring word count in the input file is TEXT-KEY-and.

- The total count of the most frequently occurring word count in the input file is 8167 times.

```
[training@quickstart ~]$ hdfs dfs -cat assign1a/part-r-00000 | grep "TEXT-KEY-"
| sort -k2 -nr | head -n 5
TEXT-KEY-and    8167
TEXT-KEY-the    7663
TEXT-KEY-to     5289
TEXT-KEY-a      4434
TEXT-KEY-of     3639
[training@quickstart ~]$
```

Figure 6: Top-5 word most occurences in file `Little_Women.txt`

- If you are modifying the codes used in Part 1 to get answers for Part 2, you should at least make changes to (Driver/Mapper/Reducer)

  => The modified component is the Mapper because it is the component that handles data sanitization, tokenization, and processing filter. In addition, for refining the results for Part 2 including filtering out common stop words including and, the, to, or converting strings to lowercase to prevent duplicates, I have to modify the internal logic within ***map()*** method

- Post the screenshot of the first page of your result in Assignment1_2.

```
1       4930
10      135
100     1
101     4
1013    1
102     2
103     2
1034    1
104     2
105     1
1057    1
106     1
1064    1
107     3
108     1
109     1
11      119
110     2
112     1
113     3
1135    1
115     2
116     2
:
```

Figure 7: Result of assign1a

- Post the complete code with comments (can be a screenshot) of which at least you need to change from Part 1 to get answers for Part 2.

```
    protected void map(LongWritable key, Text value, Context context) throws IOException, InterruptedE
        String line = value.toString().trim();
        if (line.isEmpty()) {
            return;
        }

        String[] parts = line.split("\\s+");
        if (parts.length >= 2) {
            String countValue = parts[parts.length - 1];
            boolean isNumeric = true;
            for (int i=0; i<countValue.length(); i++) {
                if (!Character.isDigit(countValue.charAt(i))) {
                    isNumeric = false;
                    break;
                }
            }

            if (isNumeric) {
                outputKey.set(countValue);
                context.write(outputKey, ONE);
            }
        }
    }
}
```

Figure 8: The Map() method

6. Explain your chain of thought in solving the given tasks. Include a flowchart in the report to explain the chain of thought step by step.

Ans: The task is solved including 3-step following:

- Execution: Ran the MapReduce job (Assign1b.jar) on Hadoop to pprocess the raw inpur text.
- Filtration: Used HDFS terminal commands combined with grep “TEXT-KEY-” to isolate text tokens from numeric records.
- Extraction: Applied a reverse numerical sort (sort -k2 -nr | head -n 1) to instantly surface the highest-frequency word and its exact count.

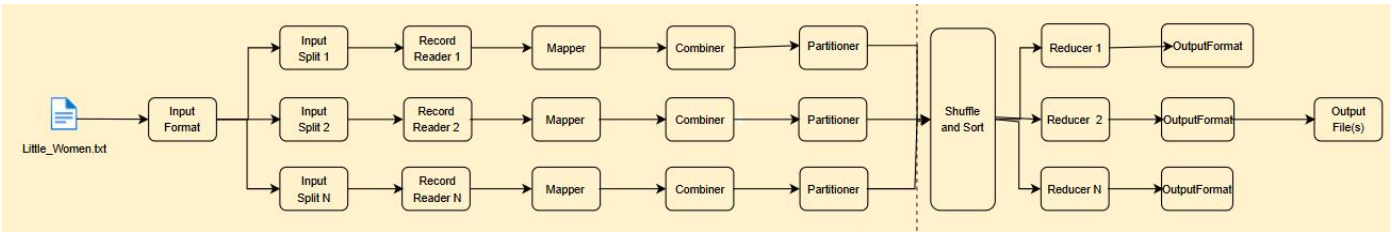


Figure 9: Progress of Assignment1 part A

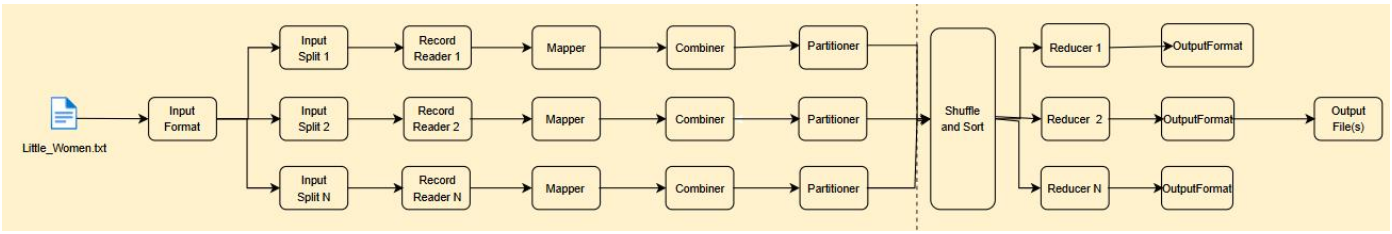


Figure 10: Progress of Assignment1 part B

------ End of Template ------